

MAIE 5532: Machine Learning Systems

Week 4 - Parallelism

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lfteris NTAFL0S, PhD

The Hong Kong University of Science and Technology

Week 4 Overview

Parallelism: From Data Centers to Microcontrollers

Learning Objectives:

- Master parallelism strategies across computational scales
- Analyze energy-efficient parallel processing techniques
- Design systems considering size constraints and parallel execution trade-offs
- Understand distributed processing challenges in ML systems

Required Readings:

- Book 1 - Volume 2: Distributed Training
- Book 1 - Volume 2: Collective Communication
- Book 1 - Volume 2: Edge Intelligence
- Book 1 - Volume 1: Model Compression
- Book 2: Chapter 16: Optimizing Energy Usage
- Book 2: Chapter 17: Optimizing Model and Binary Size

The Parallelism Spectrum



From Cloud to Edge: Different Scales, Different Challenges

The Challenge:

Modern ML systems span an enormous computational spectrum - from data center training with thousands of GPUs to inference on microcontrollers with single-digit milliwatt power budgets. Each scale presents unique parallelism opportunities and constraints.

Scale Spectrum:

- Data Center: 8-1000s of GPUs, unlimited power, high-bandwidth interconnects
- Mobile/Edge: 4-8 cores, battery constraints, moderate memory
- Microcontroller: Single core, <1mW power, <1MB memory

Real-world Impact:

A transformer model training uses data parallelism across 256 GPUs, while the same model deployed on a microcontroller must use temporal parallelism within individual operations to meet power constraints.

Session Structure

Session 1: Multi-Scale Parallelism Strategies

- Data parallelism: dataset distribution across devices
- Model parallelism: network distribution strategies
- Pipeline parallelism: : microbatching the input

Session 2: Energy-Efficient Parallel Processing

- Hybrid approaches and scaling challenges
- Microcontroller parallelism constraints
- Power consumption models in parallel systems
- Duty cycling and frequency scaling
- Cascading design for energy efficiency
- Hardware choice considerations

Session 3: Size Constraints and Memory Management

- Memory hierarchy in parallel systems
- Flash and RAM optimization strategies
- Model size vs. parallelism trade-offs
- Resource allocation in constrained environments

SESSION 1: MULTI-SCALE PARALLELISM STRATEGIES

Data Parallelism Fundamentals

Distributing Datasets for Parallel Training

Core Concept:

Data parallelism splits training datasets across multiple devices, with each device maintaining a complete copy of the model. This approach scales naturally with dataset size and provides linear speedup under ideal conditions.

Mathematical Foundation:

For N devices processing batch size B :

- Global gradient equals to the average of local gradients: $g_{global} = (1/N) \times \sum(g_i)$ where g_i is local gradient
- Equivalent to single device with batch size $N \times B$
- Maintains SGD (Stochastic Gradient Descent) mathematical properties

Device Coordination:

1. Dataset splitting into non-overlapping subsets
2. Independent forward and backward passes
3. Gradient synchronization via ring all-reduce
4. Synchronized parameter updates

Scaling Benefits:

Linear speedup possible up to communication bottleneck threshold.

Quick Review – Stochastic Gradient Descent

The core idea of SGD is to **update the model's parameters** using the **gradient** calculated from a **small, randomly selected subset of the training data**, known as a mini-batch, rather than the entire dataset. In its purest form, SGD updates parameters using only a single data point at a time, though using mini-batches is more common in practice and often referred to as SGD.

- **Mathematical Formulation:** For model parameters ϑ , learning rate η , and loss $L(\vartheta)$:
- **Standard Gradient Descent:** $\vartheta_{t+1} = \vartheta_t - \eta \nabla L(\vartheta_t)$ (uses full dataset)
- **Stochastic Gradient Descent:** $\vartheta_{t+1} = \vartheta_t - \eta \nabla L_i(\vartheta_t)$ (uses single random example i)
- **Mini-batch SGD (most common):** $\vartheta_{t+1} = \vartheta_t - \eta \nabla LB(\vartheta_t)$ - Where B is a mini-batch of examples and $\nabla LB(\vartheta_t) = (1/|B|) \sum_{i \in B} \nabla L_i(\vartheta_t)$

Algorithm Steps

1. Initialize model parameters ϑ_0 randomly
2. Shuffle the training dataset
3. For each epoch (pass through the dataset):
 - For each mini-batch B in dataset:
 - Compute gradient $\nabla LB(\vartheta_t)$
 - Update parameters: $\vartheta_{t+1} = \vartheta_t - \eta \nabla LB(\vartheta_t)$
4. Repeat for multiple epochs until the loss becomes sufficiently small or stops improving

Example Update Rule: For linear regression with MSE loss: $L(\vartheta) = (1/2n) \sum (y_i - \vartheta^T x_i)^2$

SGD update for parameter ϑ_j : $\vartheta_j \leftarrow \vartheta_j + \eta (y_i - \vartheta^T x_i) x_{ij}$

Data Parallelism Mechanics

Data Parallelism Fundamentals

Step-by-Step Execution Process

Execution Flow:

Input Data → Split into Batches → Distribute to GPUs



GPU 1: Forward Pass → Backward Pass → Compute Gradients

GPU 2: Forward Pass → Backward Pass → Compute Gradients

GPU 3: Forward Pass → Backward Pass → Compute Gradients

GPU 4: Forward Pass → Backward Pass → Compute Gradients



Gradient Synchronization (Ring All-Reduce)



Parameter Update on All Devices → Next Iteration

Communication Pattern:

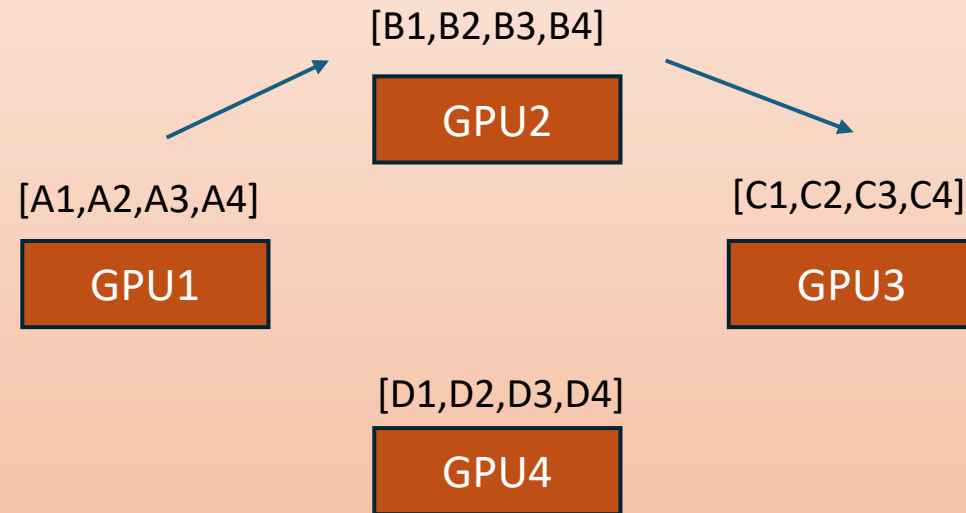
Ring all-reduce algorithm minimizes communication overhead:

- Each device communicates only with neighbors
- 7 steps for 8 devices vs. 56 for naive peer-to-peer
- Bandwidth usage scales optimally with device count

Implementation Example:

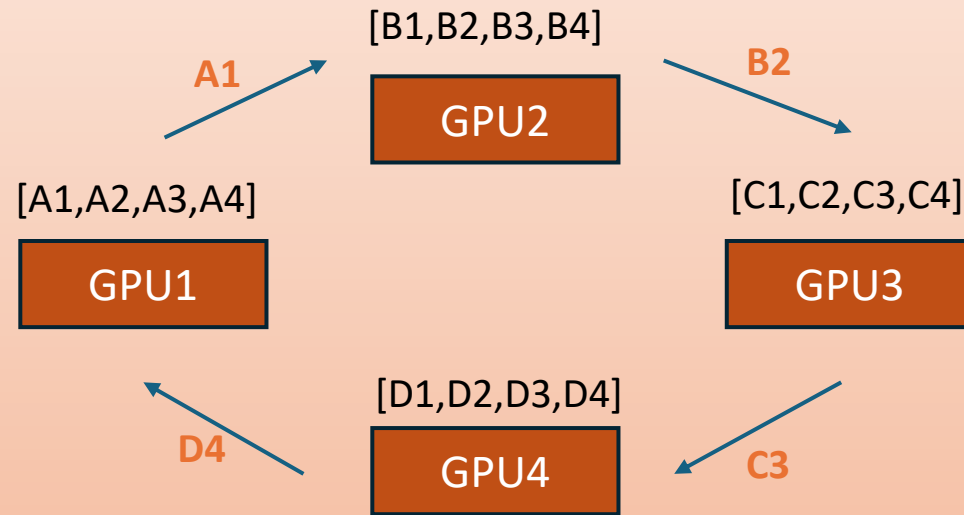
Modern frameworks like PyTorch DistributedDataParallel handle synchronization automatically.

Ring All-Reduce example



1. Initial State: Each GPU splits its gradient into 4 chunks (1 per GPU).

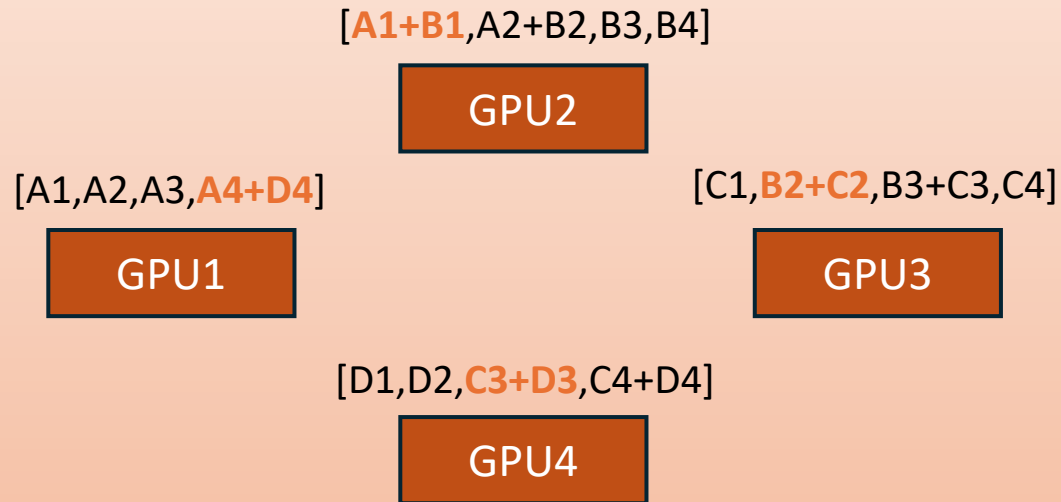
Ring All-Reduce example



“Share” Step

2. Ring Communication (Reduce – Scatter)
phase : All GPUs send chunks to their neighbors. The chunks are added (reduced).

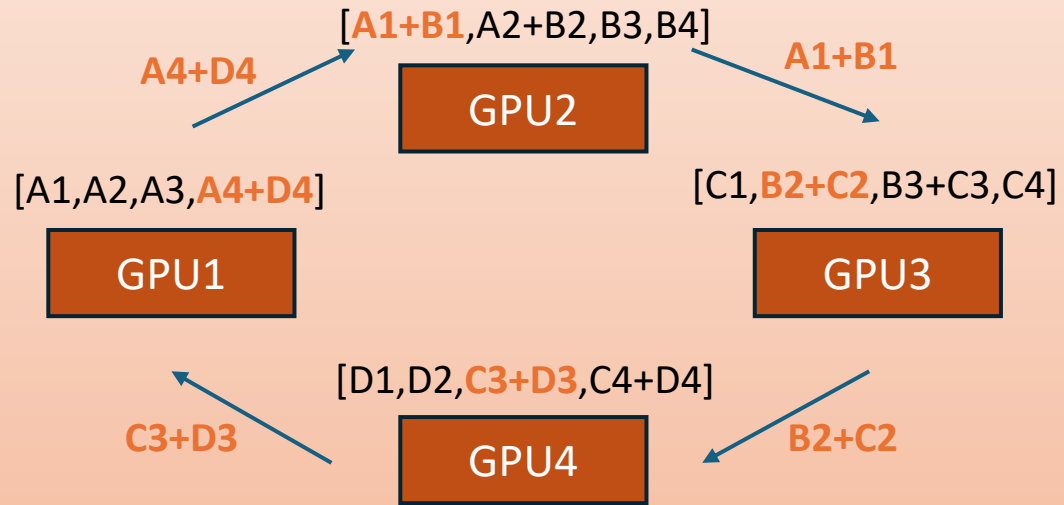
Ring All-Reduce example



“Reduce” Step

2. Ring Communication (Reduce – Scatter)
phase : All GPUs send chunks to their neighbors. The chunks are added (reduced).

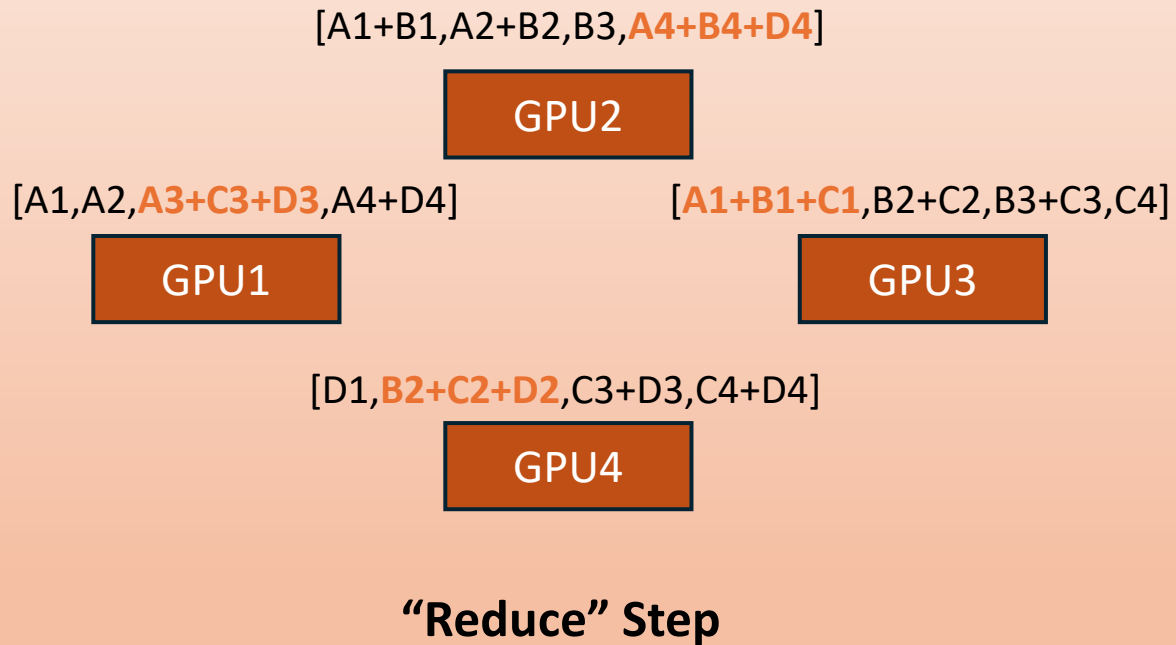
Ring All-Reduce example



"Share" Step

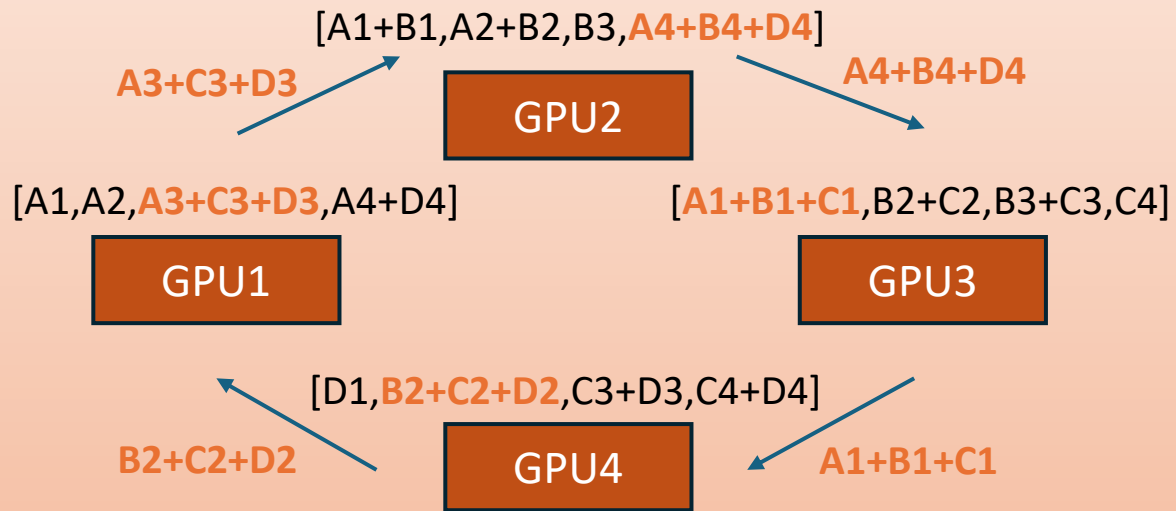
2. Ring Communication (Reduce – Scatter) phase : All GPUs send chunks to their neighbors. The chunks are added (reduced).

Ring All-Reduce example



2. Ring Communication (Reduce – Scatter) phase : All GPUs send chunks to their neighbors. The chunks are added (reduced).

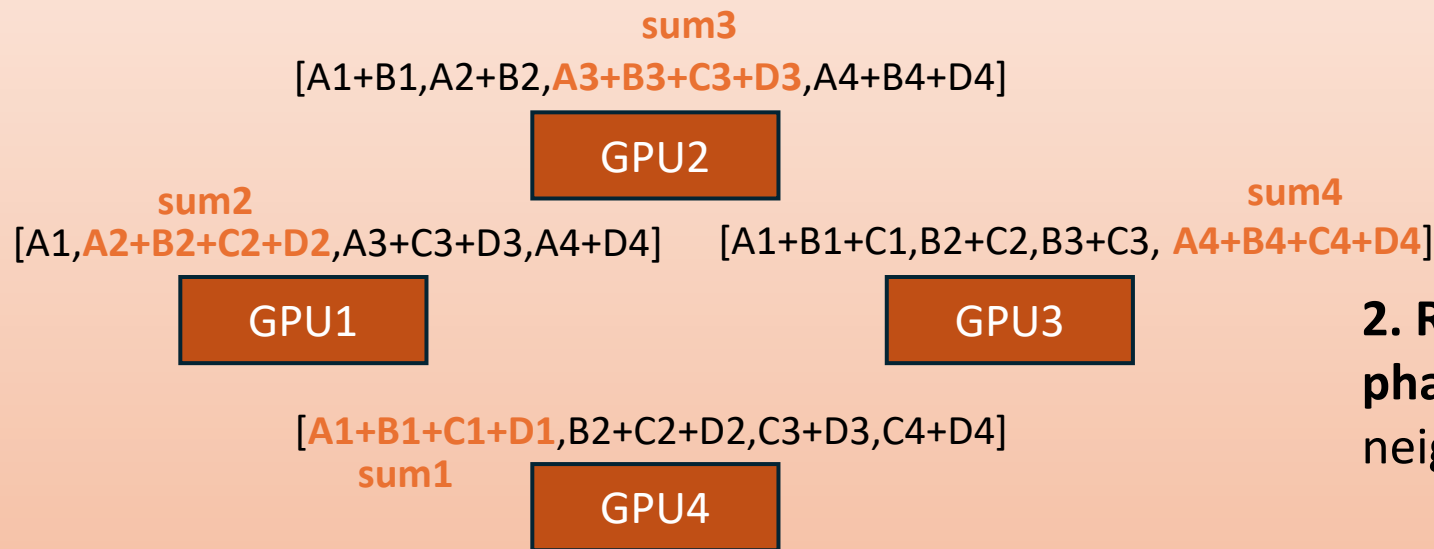
Ring All-Reduce example



“Share” Step

2. Ring Communication (Reduce – Scatter) phase : All GPUs send chunks to their neighbors. The chunks are added (reduced).

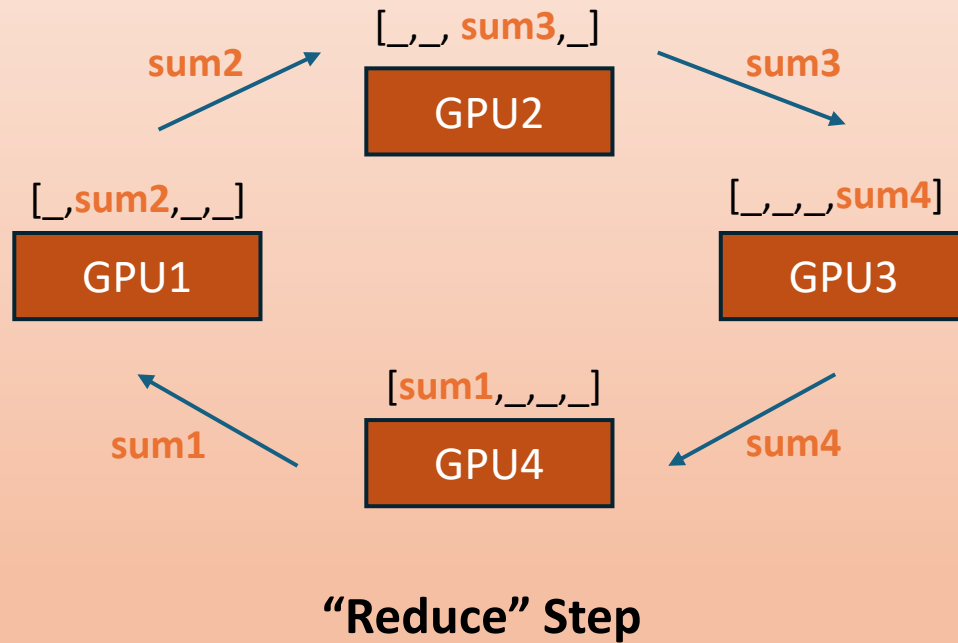
Ring All-Reduce example



“Reduce” Step

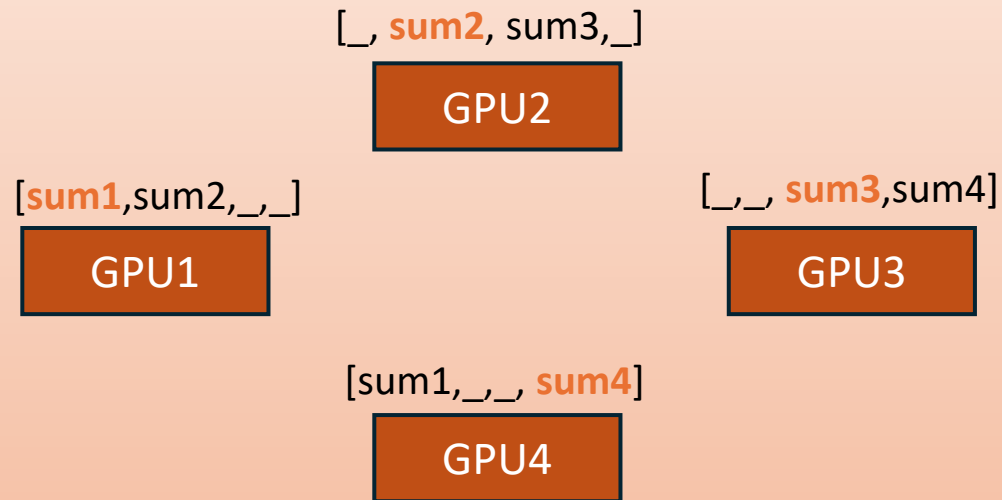
2. Ring Communication (Reduce – Scatter)
phase: All GPUs send chunks to their neighbors. The chunks are added (reduced).

Ring All-Reduce example



3. All-gather (broadcast) phase: each GPU sends its fully summed chunk to its neighbor

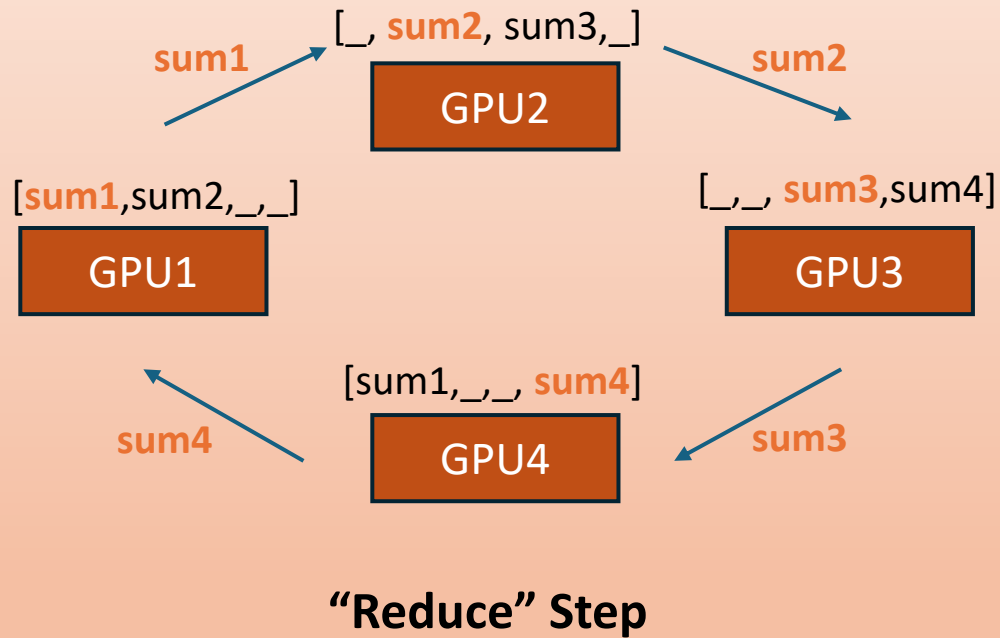
Ring All-Reduce example



“Reduce” Step

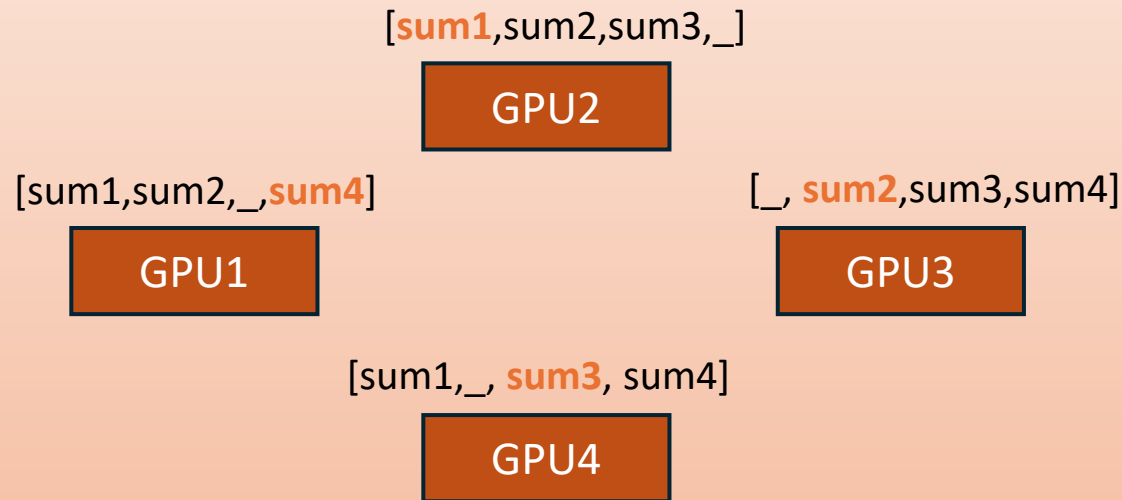
3. All-gather (broadcast) phase: each GPU sends its fully summed chunk to its neighbor

Ring All-Reduce example



3. All-gather (broadcast) phase: each GPU sends its fully summed chunk to its neighbor

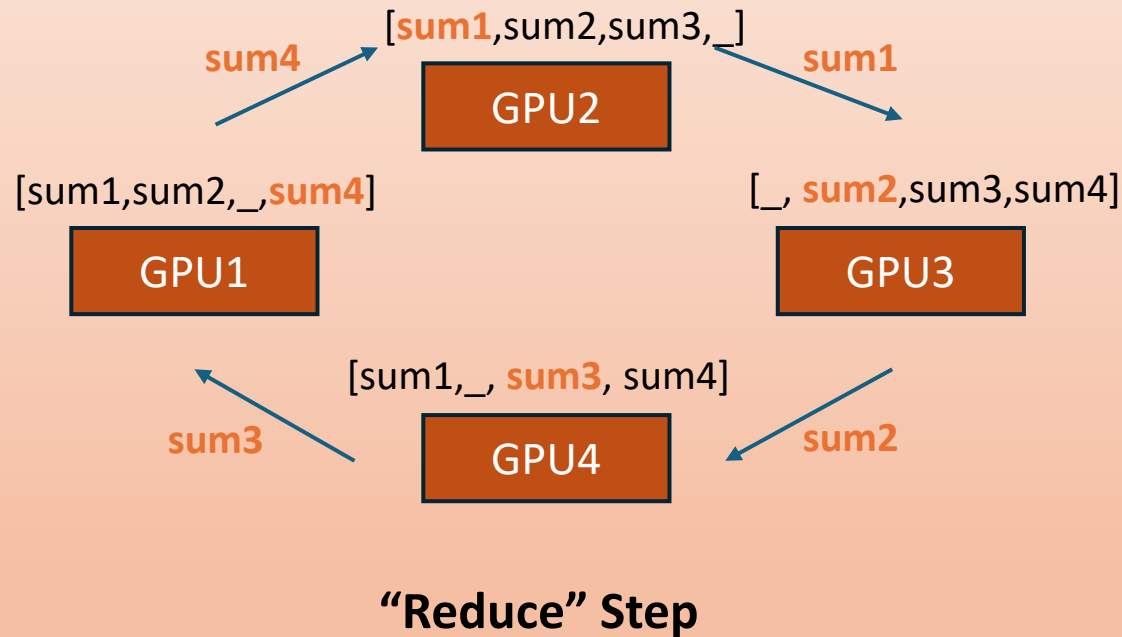
Ring All-Reduce example



“Reduce” Step

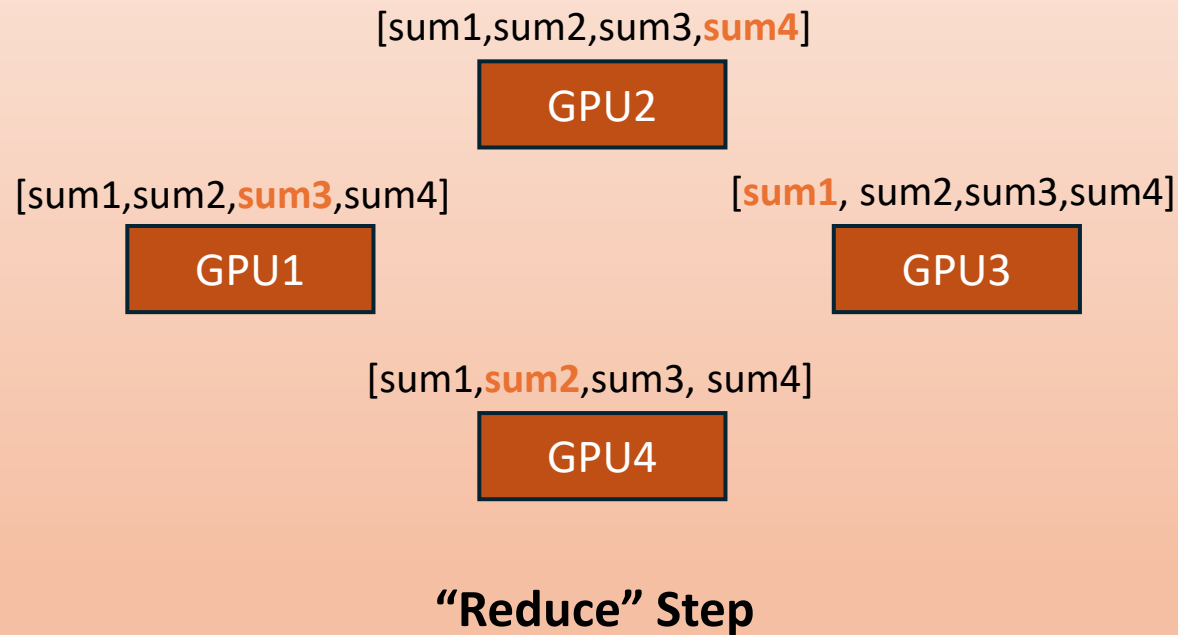
3. All-gather (broadcast) phase: each GPU sends its fully summed chunk to its neighbor

Ring All-Reduce example



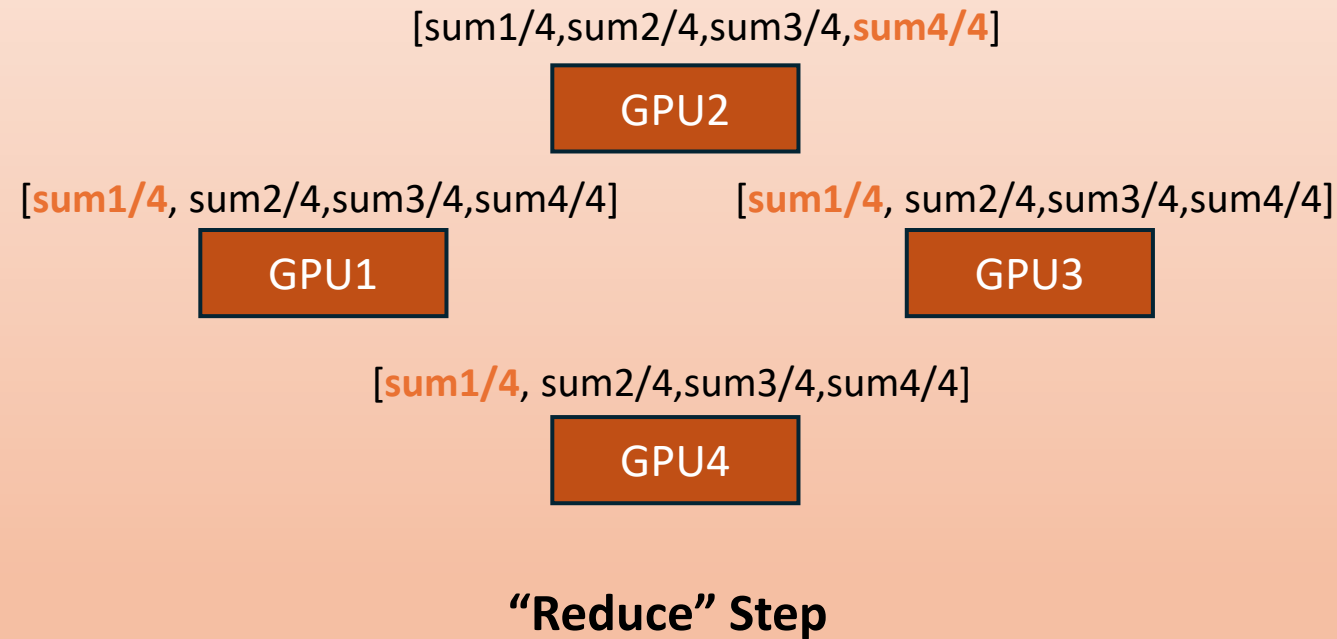
3. All-gather (broadcast) phase: each GPU sends its fully summed chunk to its neighbor

Ring All-Reduce example



3. All-gather (broadcast) phase: each GPU sends its fully summed chunk to its neighbor

Ring All-Reduce example



4. Final phase - averaging: each GPU divides its chunks by 4.

Data Parallelism Mechanics

Scaling Limitations and Communication Overhead

Communication Bottleneck:

For 1B parameter model with 8 GPUs:

- Gradient data: ~4GB per synchronization step
- High-speed interconnects required (NVLink: 900 GB/s, InfiniBand: 400 Gb/s)
- Communication time grows with model size and device count

Scaling Efficiency:

- 8 GPUs: typically 87.5% efficiency (7x speedup)
- 64 GPUs: typically 70-78% efficiency (45-50x speedup)
- Diminishing returns due to synchronization overhead

Memory Constraints:

175B parameter model requires:

350GB for FP32 parameters

Additional memory for optimizer states and activations

Total: >1TB per device requirement

Exceeds current GPU memory (80GB high-end)

Workload Imbalance:

Heterogeneous systems suffer from device speed mismatches.

Model Parallelism Strategies

Distributing Neural Networks Across Devices

When Model Parallelism is Necessary:

- Model parameters exceed single device memory
- Examples: GPT-3 (175B parameters), large vision transformers
- Memory requirement exceeds available hardware capacity

Partitioning Approaches:

1. Layer-wise: Sequential layer distribution across devices
2. Operator-level: Split operations within layers (attention heads, matrix dimensions)
3. Pipeline: Overlapped execution with microbatching

Coordination Pattern:

Forward pass: sequential data flow through device chain

Backward pass: gradient propagation in reverse order

Parameter updates: local to each device partition

Mathematical Equivalence:

Distributed computation maintains identical results to single-device execution.

Tensor Parallelism

Splits individual layers (weight matrices, tensors) across GPUs. Every GPU processes a piece of the same layer at the same time.

Pipeline Parallelism - Microbatching

Basic Pipeline Problem:

Time →

Device 1:	[Compute]	[Idle]	[Idle]	[Idle]
Device 2:	[Idle]	[Compute]	[Idle]	[Idle]
Device 3:	[Idle]	[Idle]	[Compute]	[Idle]
Device 4:	[Idle]	[Idle]	[Idle]	[Compute]

This naive "layer-wise" parallelization is inefficient because it's sequential. Only one device works at a time while the others sit idle, leading to poor hardware utilization.

Pipeline Solution with Microbatches:

Time →

Device 1:	[F ₀]	[F ₁]	[F ₂]	[F ₃]	[B ₃]	[B ₂]	[B ₁]	[B ₀]
Device 2:		[F ₀]	[F ₁]	[F ₂]	[B ₂]	[B ₁]	[B ₀]	
Device 3:			[F ₀]	[F ₁]	[B ₁]	[B ₀]		
Device 4:				[F ₀]	[B ₀]			

Splitting the input into microbatches allows devices to work simultaneously on different data chunks. This overlaps computation and drastically cuts idle time ("bubbles"), reducing them from $(m-1)$ to $(m-1)/b$ (where m is stages, b is microbatches).

F = Forward pass, B = Backward pass, subscript = microbatch number



In-Class Quiz 1 – Question 1

Given a 70B-parameter model (~280 GB in FP32) and 32 GPUs with 80 GB each, which primary constraint most strongly rules out “data parallelism only”?

- A. Insufficient batch size
- B. Model weights do not fit in a single GPU
- C. Too few data shards for workers
- D. Optimizer cannot run on GPUs



In-Class Quiz 1 – Question 2

Switching from FP32 to BF16/FP16 typically reduces weight memory by approximately what factor?

- A. 1.25×
- B. 2×
- C. 3×
- D. 4×



In-Class Quiz 1 – Question 3

With pure model parallelism (tensor/sequence sharding) across many GPUs, which overhead becomes the most critical to evaluate?

- A. Disk I/O for dataset loading
- B. All-reduce/all-gather communication between layers
- C. Gradient checkpointing cost
- D. CPU preprocessing time



In-Class Quiz 1 – Question 4

Pipeline parallelism primarily reduces which bottleneck for very deep models?

- A. Activation memory at each stage
- B. Weight precision errors
- C. GPU clock throttling
- D. Dataset size



In-Class Quiz 1 – Question 5

In pipeline parallelism, the “pipeline bubble” refers to:

- A. Extra GPU RAM used for microbatches
- B. Idle time while stages wait for other stages to produce/consume microbatches
- C. Compression artifacts during communication
- D. Gradient noise at small batch sizes



In-Class Quiz 1 – Question 6

For a hardware 32 GPUs with 80 GB memory each, which combined strategy is most commonly used to balance memory fit and throughput for very large models?

- A. Data parallelism only
- B. Model (tensor) parallelism only
- C. Hybrid: data + tensor + pipeline parallelism
- D. Pipeline parallelism only



In-Class Quiz 1 – Question 7

Which component often dominates communication volume in data parallel training of large transformer models?

- A. Parameter broadcast at startup
- B. Gradient all-reduce each step
- C. Checkpoint writing to disk
- D. Tokenization transfers from CPU



In-Class Quiz 1 – Question 8

When estimating total training time, which pair is the most important to model realistically?

- A. FLOPs per token and tokens per second per GPU (including comms)
- B. Dataset file count and shard size
- C. PCIe lane width and NVMe capacity
- D. GPU temperature and fan RPM



In-Class Quiz 1 – Question 9

If the model fits only after using activation checkpointing, what trade-off should you expect?

- A. Higher accuracy at the same compute
- B. Lower communication overhead
- C. Extra compute/time to recompute activations during backward
- D. Elimination of pipeline bubbles



In-Class Quiz 1 – Question 10

For 1T tokens target and the 32-GPU cluster, which high-level recommendation best aligns with “complete training in reasonable time” while fitting memory?

- A. Use FP32 everywhere with data parallelism only
- B. Use mixed precision, tensor+pipeline parallelism to fit/scale, plus moderate data parallel replicas
- C. Use CPU offloading for all weights and activations
- D. Train sequentially on a single GPU to avoid communication



Component Power Breakdown:

SESSION 2: ENERGY-EFFICIENT PARALLEL PROCESSING

**Power Consumption Models –
Understanding Energy Usage Across
Scales**

1. Data Center Scale:

- CPU: 500-2000mW
- GPU: 250W (training), 75W (inference)
- Memory: 5-10W per DIMM
- Network: 10-50W per high-speed port

2. Mobile/Edge Scale:

- Application processor: 1-5W
- Display: 400mW
- Radio (active): 800mW
- Memory: 100-500mW

3. Microcontroller Scale:

MCU: 1mW (Cortex-M4F at 48MHz)
Sensors: 1mW (accelerometer),
300 μ W (microphone)
Radio: 40mW (BLE), 100mW (WiFi)
Memory: Negligible for SRAM

Key Insight:

Radio communication dominates power at all scales.

Microcontroller Parallelism Constraints

SIMD stands for **Single Instruction, Multiple Data**.

It is a CPU feature that allows one instruction to perform the same operation on multiple pieces of data simultaneously, dramatically accelerating repetitive computations common in audio, video, graphics, and AI processing.

Parallelism at the Milliwatt Scale

Fundamental Limitations:

- Single core execution (no multi-threading)
- <1mW power budget
- <1MB total memory (flash + RAM)
- Limited instruction-level parallelism

Available Parallelism Types:

1. **SIMD Instructions:** ARM NEON, Helium vector processing
2. **Operator-level:** Parallel accumulation within convolutions
3. **Pipeline:** Instruction pipeline optimization
4. **Temporal:** Time-division processing strategies

Power Implications:

- Additional processing units consume power:
- ARM Cortex-M4F: 1mW at 48MHz
- Adding parallel units increases power linearly
- Must justify parallelism benefits vs. power cost

Memory Bandwidth Constraints:

- Limited memory bandwidth restricts parallel data access patterns.



CR2032 Coin Cell Battery –
Source: <https://amimarine.com/>

Hybrid Parallelism Strategies

Combining Multiple Parallelism Approaches

Hierarchical Organization - Data Parallel Groups

- **Group 1:** Model Parallel Devices [1,2,3,4]
- **Group 2:** Model Parallel Devices [5,6,7,8]
- **Group 3:** Model Parallel Devices [9,10,11,12]

Coordination Requirements:

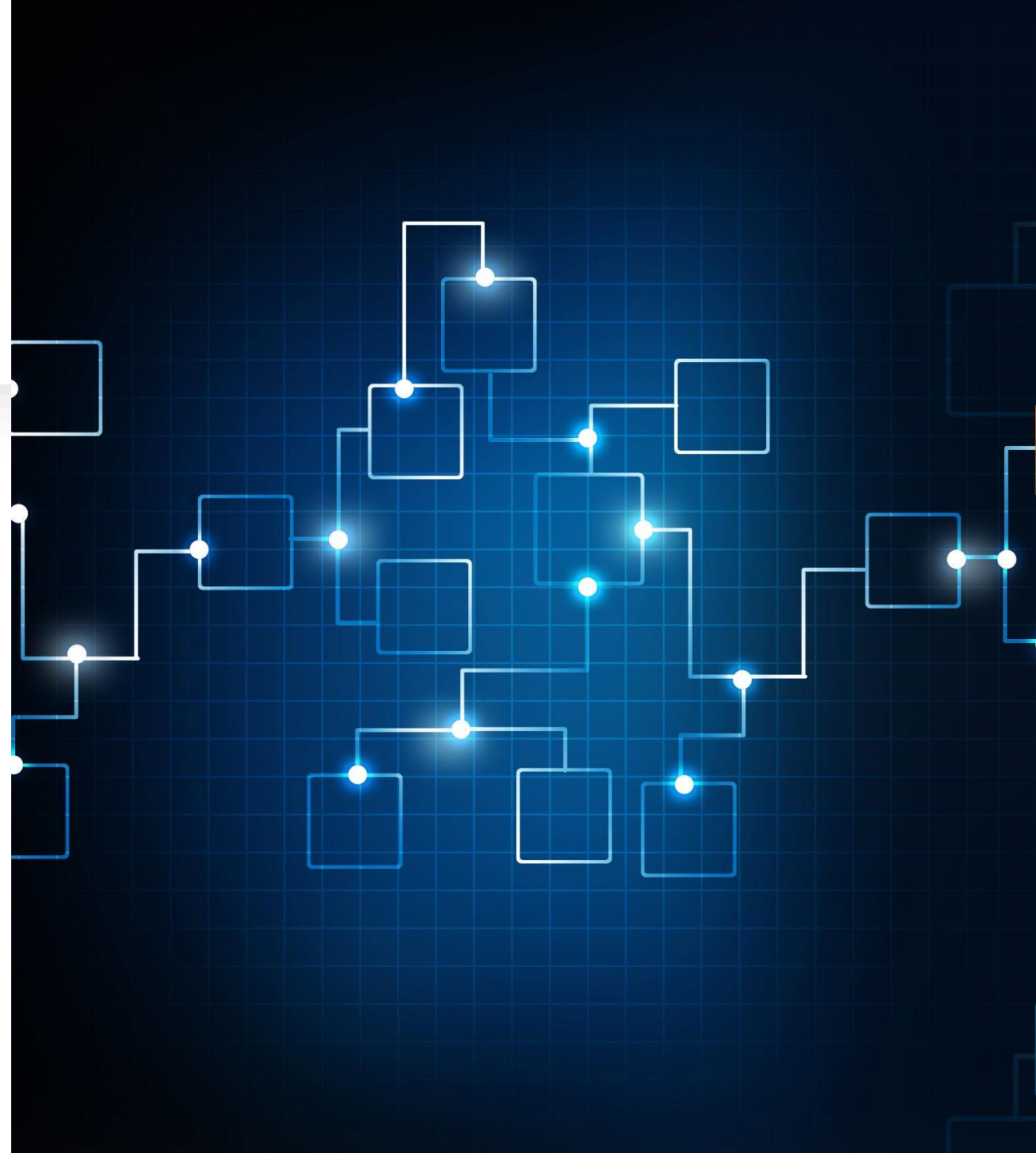
- **Intra-group:** Model parallel coordination
- **Inter-group:** Data parallel gradient synchronization
- **Pipeline:** Microbatch coordination within groups

Benefits:

- Scales both model size and dataset size
- Maximizes hardware utilization
- Adapts to hardware topology

Challenges:

- Complex communication patterns
- Increased implementation complexity
- Multiple failure modes
- Load balancing across hierarchy levels



Communication Patterns Analysis

Understanding Bandwidth Requirements

Data Parallel Communication:

- Frequency: Every training step
- Volume: Full gradient tensor (model size)
- Pattern: All-reduce across all devices
- Bandwidth: Critical for scaling

Model Parallel Communication:

- Frequency: Every forward/backward pass
- Volume: Activation tensors (batch-dependent)
- Pattern: Sequential device chain
- Latency: Critical for efficiency

Hybrid Communication:

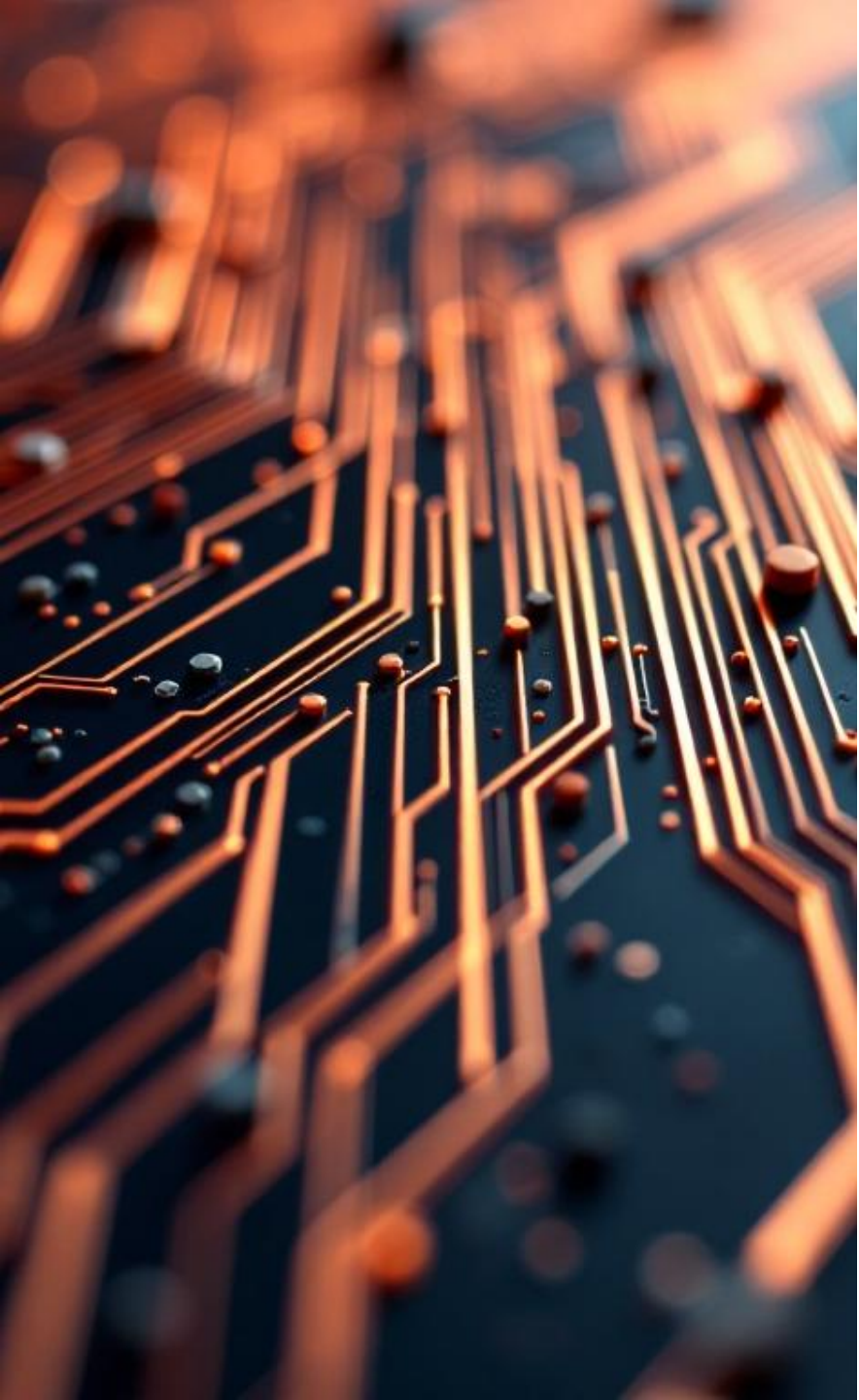
- Both patterns simultaneously
- Bandwidth requirements multiply
- Requires careful network topology design
- High-speed interconnects essential

Example Calculation:

175B parameter model, batch size 32:

- Gradients: 700GB per sync
- Activations: ~512MB per transfer
- Total bandwidth requirement: >1TB/s aggregate





Energy-Power Relationship

Fundamental Energy Concepts

Power vs. Energy:

- Power = Energy / Time (watts = joules/second)
- For battery life: Average power determines duration
- 1mW average → 2x longer life than 2mW average

Battery Capacity Examples:

- CR2032 coin battery: 2,500J (1 month at 1mW)
- AA battery: 15,000J (6 months at 1mW)
- Phone battery: ~50,000J

Energy Harvesting Potential:

- Indoor light: $10\mu\text{W}/\text{cm}^2$
- Outdoor light: $10\text{mW}/\text{cm}^2$
- Industrial thermal: $1\text{-}10\text{mW}/\text{cm}^2$
- Vibration: $10\text{-}100\mu\text{W}/\text{cm}^2$

Design Implication:

Only outdoor solar and industrial thermal currently practical for self-powered ML devices.

Duty Cycling Strategies

Trading Computation Time for Power

Basic Duty Cycling:

```
void low_power_inference_loop() {  
    while(1) {  
        // Active period  
        digitalWrite(LED_PIN, HIGH);  
        run_ml_inference();  
        process_results();  
  
        // Sleep period  
        digitalWrite(LED_PIN, LOW);  
        sleep_mode(SLEEP_DURATION);  
    }  
}
```

Power Reduction Calculation:

- Active: 10mW for 100ms every 1000ms
- Sleep: 0.1mW for 900ms every 1000ms
- Average: $(10 \times 0.1 + 0.1 \times 0.9) = 1.09\text{mW}$
- vs. continuous 10mW operation

Frequency Scaling:

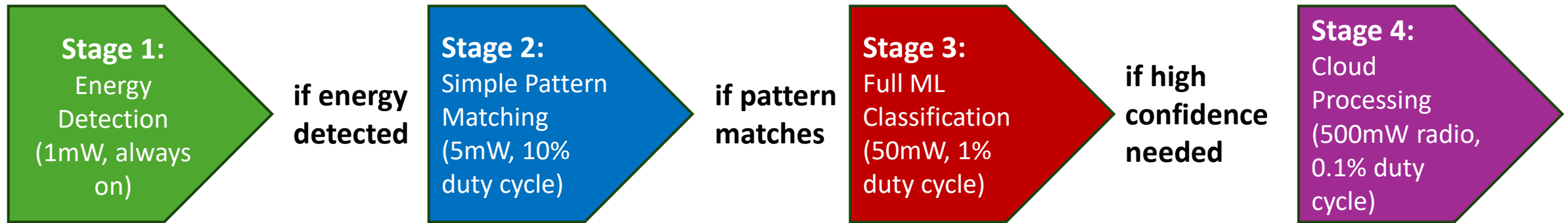
Lower clock frequency reduces power quadratically:

- 48MHz: 2mW
- 24MHz: 1.2mW
- 8MHz: 0.4mW

Trade computation speed for power savings

Cascading Design Principles

Multi-Stage Processing for Efficiency



Average Power Calculation:

- Stage 1: $1\text{mW} \times 100\% = 1.0\text{mW}$
- Stage 2: $5\text{mW} \times 10\% = 0.5\text{mW}$
- Stage 3: $50\text{mW} \times 1\% = 0.5\text{mW}$
- Stage 4: $500\text{mW} \times 0.1\% = 0.5\text{mW}$
- **Total: 2.5mW average**

Key Principles:

Each stage has higher accuracy but higher power
Later stages activate only when needed
False positive rate determines duty cycle



In-Class Exercise 1 (15 minutes)

Energy-Efficient System Design

Scenario: Smart Doorbell with Person Detection

Requirements:

- Battery life: 6 months minimum
- Person detection accuracy: >95%
- Real-time response: <500ms when person present
- False positive rate: <1% (to avoid unnecessary alerts)

Available Components:

- Camera sensor: 1mW at 1 FPS, 10mW at 10 FPS
- MCU: 1mW at 48MHz, 0.5mW at 24MHz, 0.1mW sleep
- Radio: 40mW active, 1mW standby

Available models:

- Tiny motion detector: 1M operations, 85% accuracy
- Person classifier: 60M operations, 95% accuracy

Your Task:

1. Design cascading system architecture
2. Calculate power budget for 6-month battery life
3. Determine duty cycles for each stage
4. Estimate overall system accuracy
5. Verify real-time response requirements

Hardware Choice for Energy Efficiency

Selecting Components for Power Optimization

Microcontroller Selection Criteria:

Power efficiency comparison

STM32L4 (Ultra-low power):

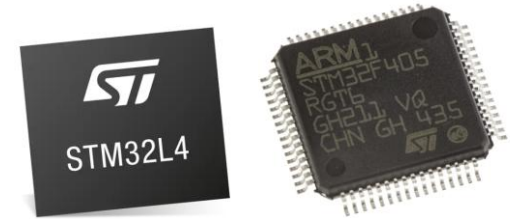
- Active: 0.1mW at 80MHz
- Sleep: 0.002mW
- Optimized for battery applications

STM32F4 (Performance):

- Active: 2mW at 168MHz
- Sleep: 0.1mW
- Higher performance, higher power

Selection depends on:

- Required compute performance
- Duty cycle pattern
- Available sleep modes



STM32L4 and STM32F4 Microcontrollers
Sources: mouser.hk and hken.rs-online.com

Memory Technology Trade-offs:

- SRAM: Fast, low power, limited capacity
- DRAM: Higher capacity, higher power consumption
- Flash: Non-volatile, moderate power, read-only

Sensor Selection:

Choose sensors with multiple power modes and appropriate sample rates for application requirements.

SRAM is static memory that is faster and simpler but less dense and more expensive, while **DRAM** is dynamic memory that is denser and cheaper but slower and requires constant refreshing.

In the context of microcontrollers, SRAM is the scarce, fast on-chip memory the CPU uses directly, while DRAM is the larger, slower external memory used in more powerful systems.

Parallel Processing Power Models

Energy Implications of Parallelism

Sequential vs. Parallel Power:

Sequential Execution:

- Single core: 1mW
- Execution time: 100ms
- Energy: 0.1mJ

Parallel Execution (4 cores):

- Four cores: $4 \times 1\text{mW} = 4\text{mW}$
- Execution time: 25ms
- Energy: 0.1mJ (same total energy)

Power-Performance Trade-offs:

- Parallelism reduces latency without changing total energy
- May enable more aggressive duty cycling
- Additional hardware increases baseline power
- Must consider parallel efficiency losses

SIMD Power Benefits:

Single instruction, multiple data operations:

- Process 4 values with 1.2× power of single value
- 3.3× energy efficiency improvement
- Available on ARM Cortex-M4 and higher

Dynamic Voltage and Frequency Scaling

Runtime Power Optimization

DVFS Principles:

$$\text{Power} \propto \text{Voltage}^2 \times \text{Frequency}$$

- Reducing frequency linearly reduces power
- Reducing voltage quadratically reduces power
- Must maintain voltage above threshold for frequency

Application Scenarios:

- Adapt to battery level
- Respond to thermal conditions
- Balance performance vs. battery life based on usage patterns

```
void adaptive_inference() {  
  if (battery_level > 80%) {  
    set_cpu_frequency(48_MHz); // Full performance  
  } else if (battery_level > 20%) {  
    set_cpu_frequency(24_MHz); // Balanced mode  
  } else {  
    set_cpu_frequency(8_MHz); // Conservation mode  
  }  
  
  run_inference();  
}
```



Session 3

**Size Constraints and
Memory Management**

A thin vertical line on the right side of the slide, colored blue at the top and transitioning to orange at the bottom.



Memory Hierarchy in Constrained Systems

Understanding Storage and Memory Limitations

Typical Embedded Memory Architecture:

Flash Memory (Read-Only Storage):

- Operating System: 50-200KB
- TensorFlow Lite Micro: 20-100KB
- Model Weights: 10KB-1MB
- Application Code: 10-50KB
- Available: Remaining capacity

SRAM (Modifiable Memory):

- Operating System: 2-10KB
- TensorFlow Lite Arena: 5-200KB
- Application Buffers: 1-20KB
- Stack: 1-5KB

Memory Constraints Impact:

- Flash: Programs + models must fit in read-only storage
- SRAM: All runtime data must fit simultaneously
- No virtual memory or swap space available
- Memory allocation failures cause system crashes

Flash Usage Estimation

Calculating Storage Requirements - Component Size Breakdown

1. Operating System:

- Minimal RTOS configuration
- FreeRTOS_minimal: ~15KB
- FreeRTOS_with_USB: ~50KB
- FreeRTOS_with_networking: ~200KB

2. TensorFlow Lite for Microcontrollers:

- Framework size depends on operations used
- Core_framework: 20KB
- With_all_operations: 100KB
- Optimized_for_specific_model: 30-50KB

3. Model Size Calculation:

Weight counting example:

- Dense_layer: $\text{input_size} \times \text{output_size} \times \text{bytes_per_weight}$
- Conv2D_layer: $\text{kernel_h} \times \text{kernel_w} \times \text{input_channels} \times \text{output_channels} \times \text{bytes_per_weight}$

Quantization impact

- FP32_weights: 4 bytes each
- INT8_weights: 1 byte each (75% reduction)

An **RTOS (Real-Time Operating System)** is a lightweight OS designed for **deterministic timing** and **fast task scheduling** in embedded systems. Unlike general-purpose OSes (e.g., Linux), an RTOS guarantees **hard or soft real-time performance**, making it ideal for time-critical applications like robotics, automotive control, and IoT edge devices.

FreeRTOS is a popular, open-source, real-time operating system (RTOS) designed for embedded systems, particularly microcontrollers.

RAM Usage Analysis

```
// Static allocation approach
uint8_t tensor_arena[50 * 1024]; // 50KB arena

tflite::MicroInterpreter interpreter(
    model, resolver, tensor_arena,
    sizeof(tensor_arena), error_reporter);

// Check if arena is sufficient
if (interpreter.AllocateTensors() != kTfLiteOk) {
    // Arena too small - increase size
}
```

Dynamic Memory Requirements

TensorFlow Lite Memory Arena:

The arena holds temporary activation tensors during inference

Arena size calculation:

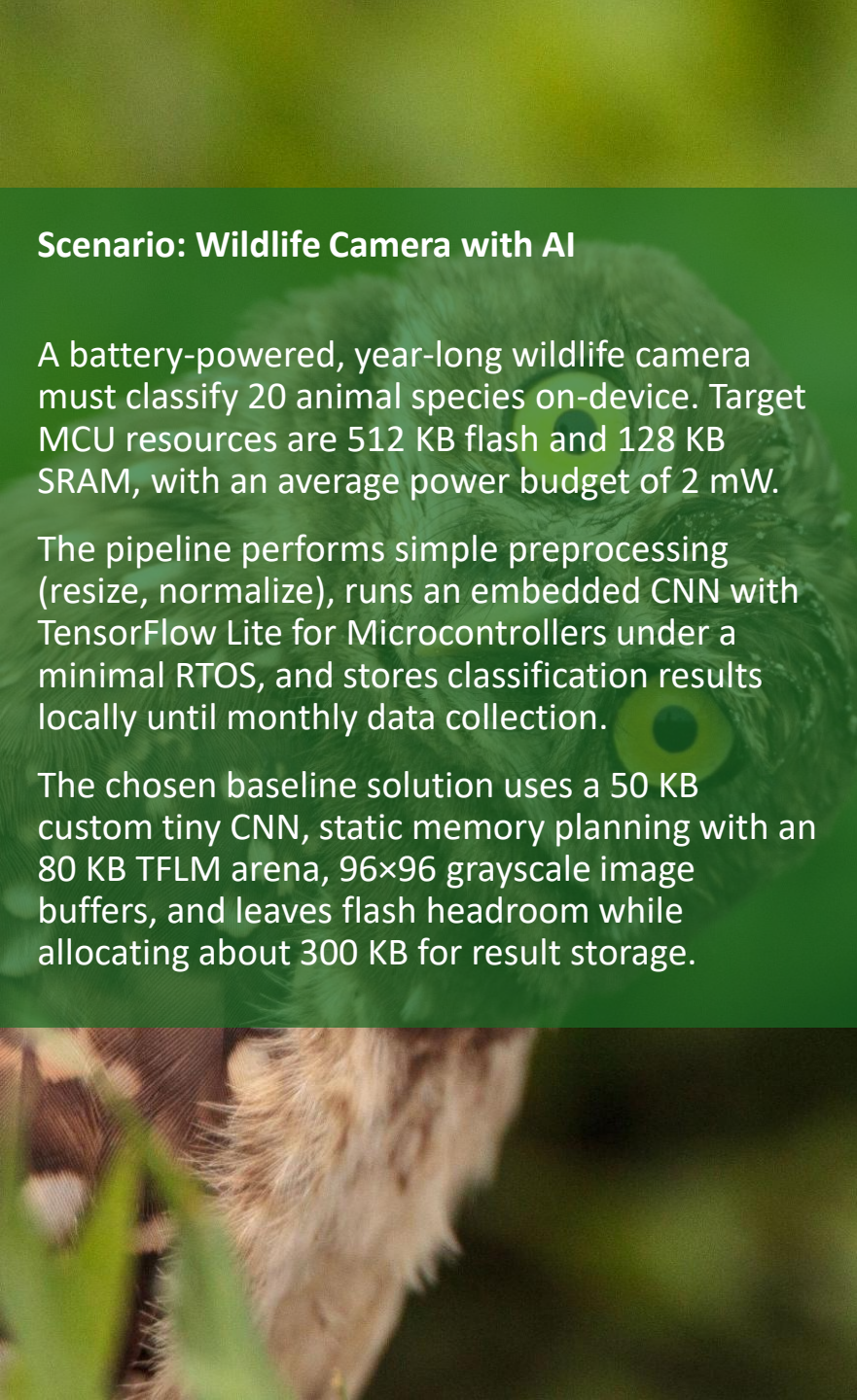
- `arena_size = max_simultaneous_tensor_memory`

Typical requirements by model type:

- **Simple_dense_model**: 5-20KB
- **Small_CNN**: 20-100KB
- **Efficient_transformer**: 50-200KB

Application Memory:

- Input/output buffers
- Preprocessing workspace
- Communication buffers
- Business logic variables



Scenario: Wildlife Camera with AI

A battery-powered, year-long wildlife camera must classify 20 animal species on-device. Target MCU resources are 512 KB flash and 128 KB SRAM, with an average power budget of 2 mW.

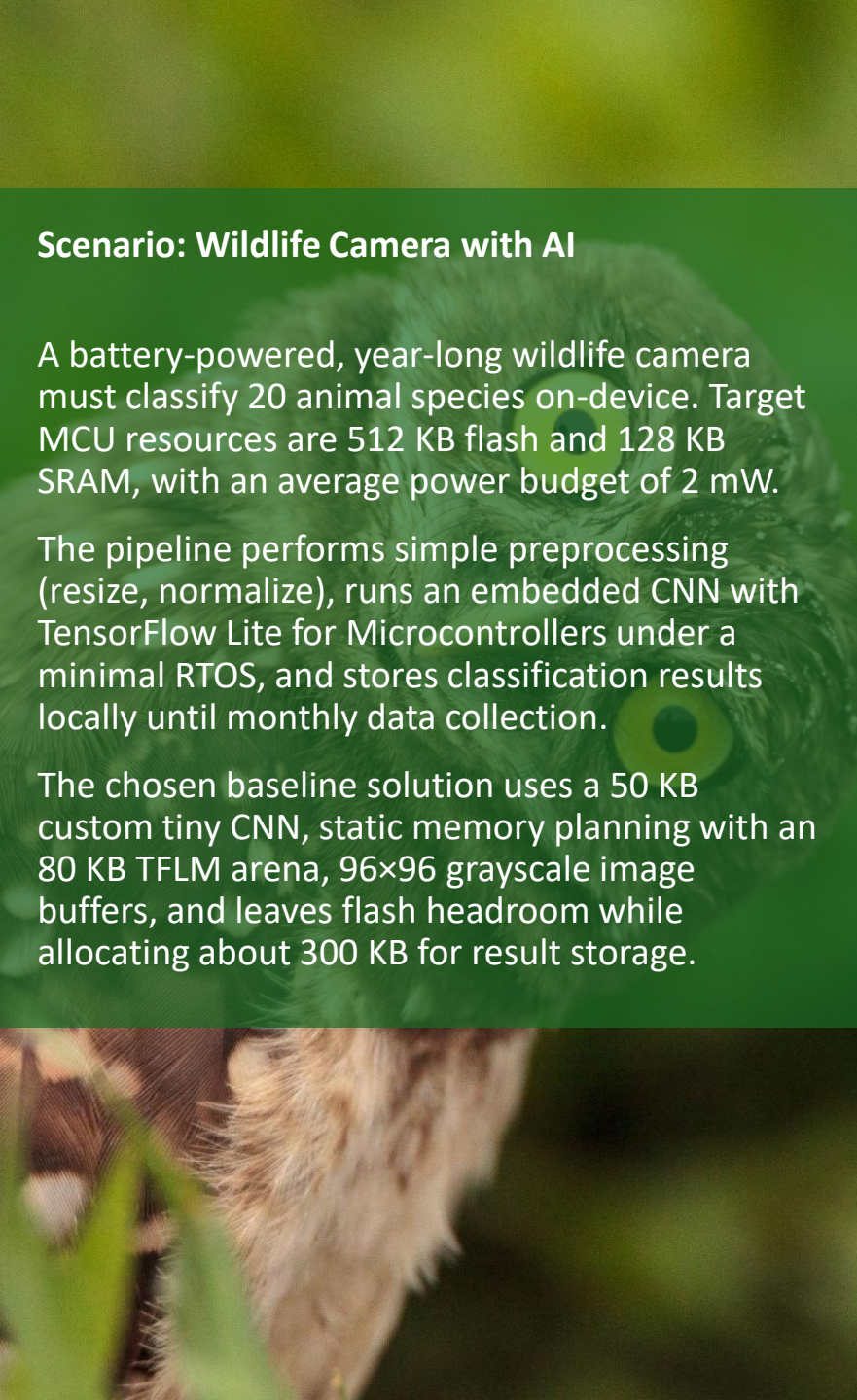
The pipeline performs simple preprocessing (resize, normalize), runs an embedded CNN with TensorFlow Lite for Microcontrollers under a minimal RTOS, and stores classification results locally until monthly data collection.

The chosen baseline solution uses a 50 KB custom tiny CNN, static memory planning with an 80 KB TFLM arena, 96×96 grayscale image buffers, and leaves flash headroom while allocating about 300 KB for result storage.

In-Class Quiz 2 – Question 1

Which model best satisfies the constraints while leaving sufficient flash for other components and storage?

- A. MobileNet-v3 small (250 KB, 95% accuracy)
- B. Custom tiny CNN (50 KB, 88% accuracy)
- C. Ultra-tiny classifier (10 KB, 75% accuracy)
- D. None of the above



Scenario: Wildlife Camera with AI

A battery-powered, year-long wildlife camera must classify 20 animal species on-device. Target MCU resources are 512 KB flash and 128 KB SRAM, with an average power budget of 2 mW.

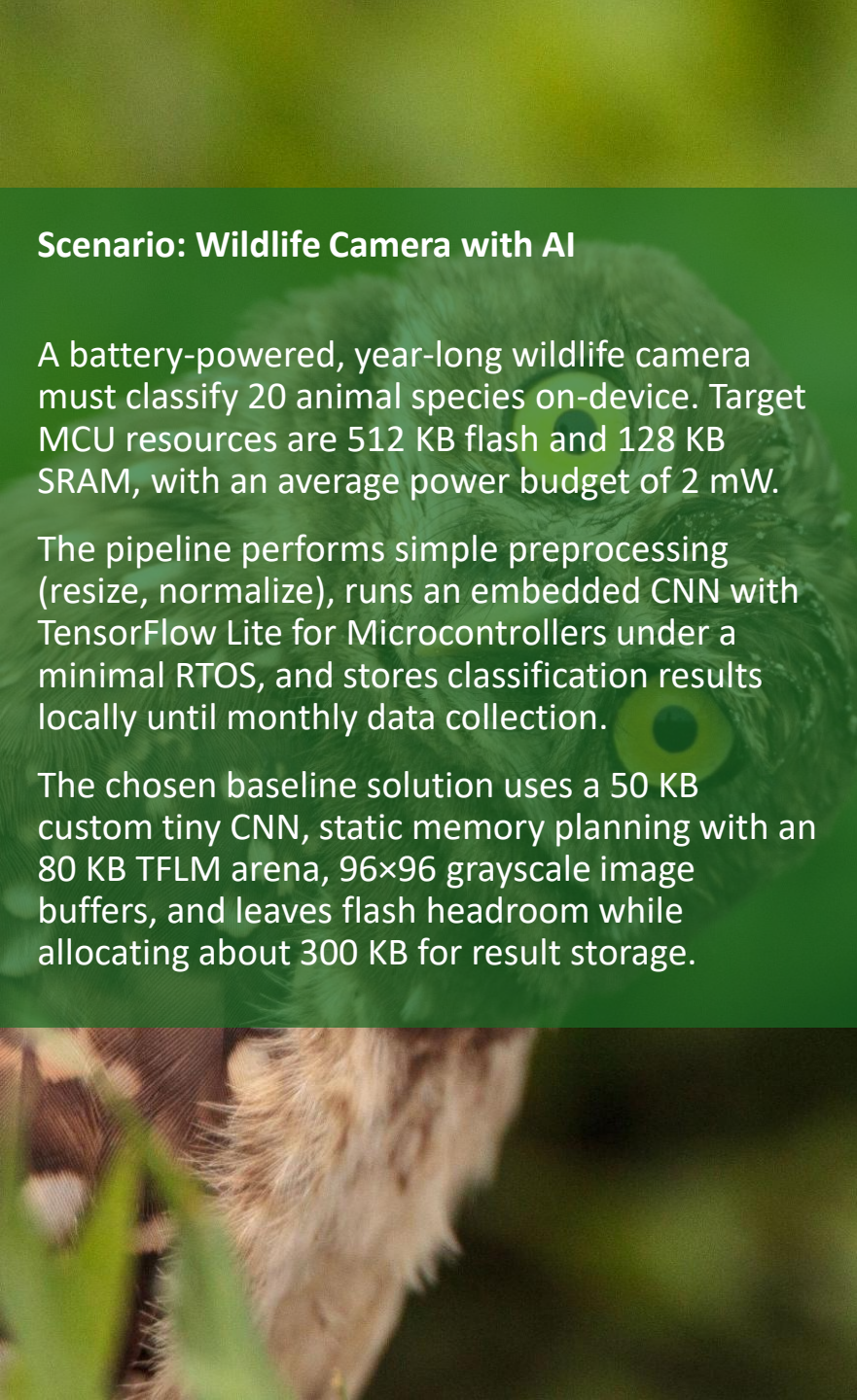
The pipeline performs simple preprocessing (resize, normalize), runs an embedded CNN with TensorFlow Lite for Microcontrollers under a minimal RTOS, and stores classification results locally until monthly data collection.

The chosen baseline solution uses a 50 KB custom tiny CNN, static memory planning with an 80 KB TFLM arena, 96×96 grayscale image buffers, and leaves flash headroom while allocating about 300 KB for result storage.

In-Class Quiz 2 – Question 2

In the final flash budget, which item reserves the largest portion of flash?

- A. TensorFlow Lite Micro runtime
- B. Image preprocessing code
- C. Result storage for one year of data
- D. FreeRTOS (minimal)



Scenario: Wildlife Camera with AI

A battery-powered, year-long wildlife camera must classify 20 animal species on-device. Target MCU resources are 512 KB flash and 128 KB SRAM, with an average power budget of 2 mW.

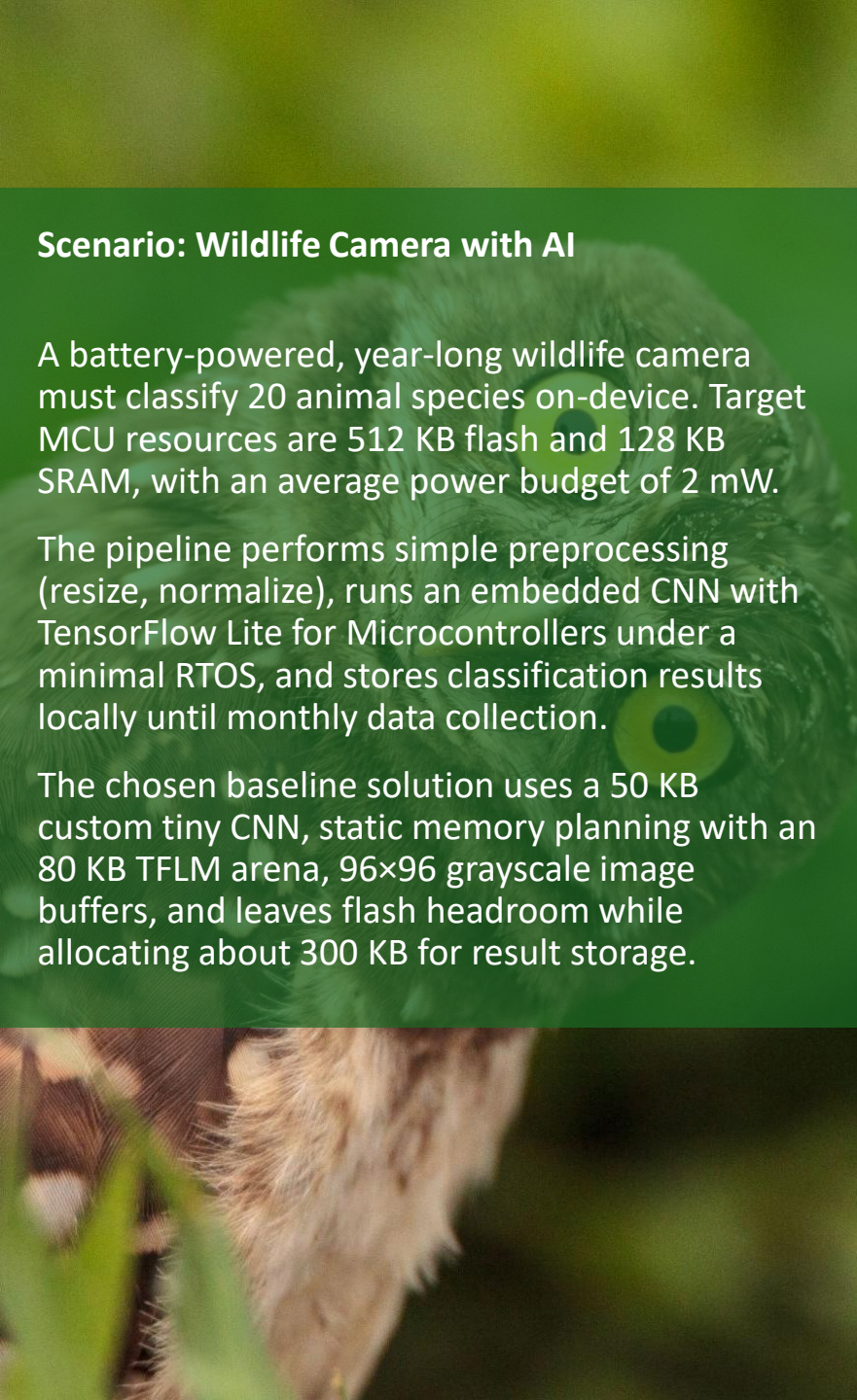
The pipeline performs simple preprocessing (resize, normalize), runs an embedded CNN with TensorFlow Lite for Microcontrollers under a minimal RTOS, and stores classification results locally until monthly data collection.

The chosen baseline solution uses a 50 KB custom tiny CNN, static memory planning with an 80 KB TFLM arena, 96×96 grayscale image buffers, and leaves flash headroom while allocating about 300 KB for result storage.

In-Class Quiz 2 – Question 3

What is the total flash usage in the final plan?

- A. 470 KB
- B. 492 KB
- C. 512 KB
- D. 520 KB



Scenario: Wildlife Camera with AI

A battery-powered, year-long wildlife camera must classify 20 animal species on-device. Target MCU resources are 512 KB flash and 128 KB SRAM, with an average power budget of 2 mW.

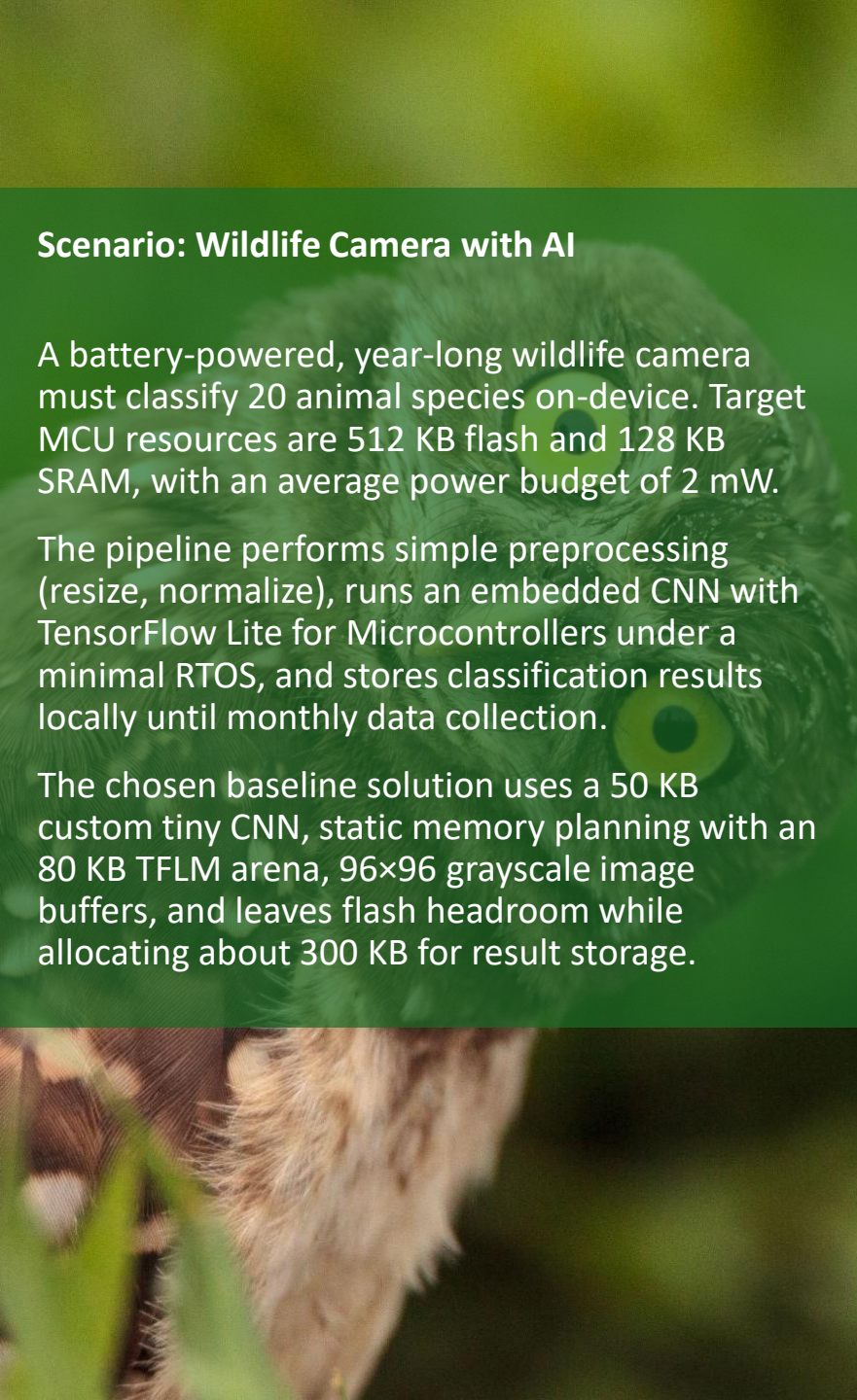
The pipeline performs simple preprocessing (resize, normalize), runs an embedded CNN with TensorFlow Lite for Microcontrollers under a minimal RTOS, and stores classification results locally until monthly data collection.

The chosen baseline solution uses a 50 KB custom tiny CNN, static memory planning with an 80 KB TFLM arena, 96×96 grayscale image buffers, and leaves flash headroom while allocating about 300 KB for result storage.

In-Class Quiz 2 – Question 4

Which single SRAM component is the largest in the allocation?

- A. Stack
- B. Image buffers (96×96 grayscale)
- C. FreeRTOS data
- D. TensorFlow arena



Scenario: Wildlife Camera with AI

A battery-powered, year-long wildlife camera must classify 20 animal species on-device. Target MCU resources are 512 KB flash and 128 KB SRAM, with an average power budget of 2 mW.

The pipeline performs simple preprocessing (resize, normalize), runs an embedded CNN with TensorFlow Lite for Microcontrollers under a minimal RTOS, and stores classification results locally until monthly data collection.

The chosen baseline solution uses a 50 KB custom tiny CNN, static memory planning with an 80 KB TFLM arena, 96×96 grayscale image buffers, and leaves flash headroom while allocating about 300 KB for result storage.

In-Class Quiz 2 – Question 5

What is the total SRAM usage, and does it meet the constraint?

- A. 118 KB; exceeds the limit
- B. 123 KB; meets the limit
- C. 128 KB; meets the limit
- D. 130 KB; exceeds the limit

Model Size Optimization Techniques

Operation Selection

```
// Remove unused operations from resolver
tflite::MicroMutableOpResolver<4> resolver;
resolver.AddBuiltin(tflite::BuiltinOperator_CONV_2D,
                    tflite::ops::micro::Register_CONV_2D());
resolver.AddBuiltin(tflite::BuiltinOperator_DEPTHWISE_CONV_2D,
                    tflite::ops::micro::Register_DEPTHWISE_CONV_2D());
resolver.AddBuiltin(tflite::BuiltinOperator_FULLY_CONNECTED,
                    tflite::ops::micro::Register_FULLY_CONNECTED());
resolver.AddBuiltin(tflite::BuiltinOperator_SOFTMAX,
                    tflite::ops::micro::Register_SOFTMAX());

// Each unused operation removed saves 1-5KB
```

Reducing Memory Footprint

Quantization Strategies:

- Weight quantization impact:
- FP32_model: 200KB
- INT8_quantized: 50KB (75% reduction)
- Mixed precision: 100KB (optimized critical paths)

Architecture Optimization:

- Replace dense layers with global average pooling
- Use depthwise separable convolutions
- Optimize channel counts for hardware alignment

Memory Access Optimization

3. Memory Pool Management:

- Pre-allocate buffers to avoid runtime allocation overhead and fragmentation.

Efficient Data Movement Patterns

1. Cache-Friendly Access Patterns:

```
// Poor memory access (random pattern)
for (int c = 0; c < channels; c++) {
    for (int h = 0; h < height; h++) {
        for (int w = 0; w < width; w++) {
            output[c][h][w] = process(input[c][h][w]);
        }
    }
}

// Better memory access (sequential pattern)
for (int h = 0; h < height; h++) {
    for (int w = 0; w < width; w++) {
        for (int c = 0; c < channels; c++) {
            output[h][w][c] = process(input[h][w][c]);
        }
    }
}
```

2. In-Place Operations:

```
// Minimize memory usage with in-place processing
void relu_inplace(float* data, int size) {
    for (int i = 0; i < size; i++) {
        data[i] = fmaxf(0.0f, data[i]);
    }
}
```

Balancing Parallelism and Memory

Trade-offs in Constrained Environments

Parallelism Memory Overhead:

```
// Sequential processing
uint8_t single_buffer[IMAGE_SIZE];
process_image(single_buffer);

// Parallel processing
uint8_t buffer_array[NUM_THREADS][IMAGE_SIZE];
for (int t = 0; t < NUM_THREADS; t++) {
    process_image_parallel(buffer_array[t], t);
}
// Memory usage: NUM_THREADS × larger
```

SIMD vs. Memory Trade-offs:

- SIMD operations require aligned data
- May need padding that increases memory usage
- Benefit: Process multiple elements per instruction
- Cost: Additional memory layout requirements

Temporal vs. Spatial Parallelism:

- Temporal: Reuse memory, process sequentially
- Spatial: Use more memory, process in parallel
- Choice depends on memory budget vs. latency requirements

Code Size Optimization

Minimizing Executable Footprint

Measuring Code Size:

```
# Build and measure
arm-none-eabi-objcopy app.elf app.bin -O binary
ls -l app.bin # Shows actual flash usage

# Analyze size breakdown
arm-none-eabi-nm -S --size-sort app.elf | tail -20
```

ELF stands for **Executable and Linkable Format**. It is the standard file format for executables, object code, and libraries on Unix-like operating systems, including those running on microcontrollers. When you compile your C/C++ code for a microcontroller, the compiler initially produces an **ELF file** (e.g., app.elf). This file contains not only the raw machine code that will run on the chip but also extra metadata like debug symbols (function and variable names), section headers, and memory layout information.

Function-Level Analysis:

```
# Identify large functions
arm-none-eabi-objdump -S app.o | grep "function_name" -A 50
```

Optimization Strategies:

- Use -Os compiler flag for size optimization
- Remove debug symbols from production builds
- Enable link-time optimization (LTO)
- Manually optimize large functions identified by analysis

Framework Tuning:

```
// Reduce framework constants
#define TFLITE_REGISTRATIONS_MAX 16 // Default: 128
#define kStackDataAllocatorSize 2048 // Adjust for application
```

Resource Allocation Strategies

Dynamic vs. Static Memory Management

Static Allocation Benefits:

```
// Predictable memory usage
static uint8_t tensor_arena[ARENA_SIZE];
static uint8_t input_buffer[INPUT_SIZE];
static uint8_t output_buffer[OUTPUT_SIZE];

// No runtime allocation failures
// No memory fragmentation
// Deterministic memory layout
```

Dynamic Allocation Challenges:

- Fragmentation in limited memory
- Allocation failure handling complexity
- Unpredictable memory usage patterns
- Higher runtime overhead

Hybrid Approach:

```
// Static allocation for critical paths
static uint8_t ml_arena[ML_ARENA_SIZE];

// Small dynamic pool for temporary needs
uint8_t dynamic_pool[SMALL_POOL_SIZE];
simple_allocator_t allocator;
init_allocator(&allocator, dynamic_pool, sizeof(dynamic_pool));
```

Performance Measurement in Constrained Systems

Profiling with Limited Resources

Timing Measurement Techniques:

```
// GPIO pin profiling
void profile_inference() {
    digitalWrite(PROFILE_PIN, HIGH);
    interpreter->Invoke();
    digitalWrite(PROFILE_PIN, LOW);
    // Measure with logic analyzer
}

// Timer-based profiling
uint32_t start_time = micros();
interpreter->Invoke();
uint32_t duration = micros() - start_time;
```



GPIO stands for **General-Purpose Input/Output**. It's a programmable pin on a microcontroller that can be controlled by software to either read a digital signal (Input) or set a digital output voltage (Output). Image source: electronicwings.com

Memory Usage Profiling:

```
// Stack usage measurement
void check_stack_usage() {
    extern char _stack_end;
    char stack_var;
    uint32_t stack_used = &_stack_end - &stack_var;
    printf("Stack used: %lu bytes\n", stack_used);
}

// Heap usage (if using dynamic allocation)
void check_heap_usage() {
    struct mallinfo info = mallinfo();
    printf("Heap used: %d bytes\n", info.uordblks);
}
```

Energy Measurement:

Use external measurement equipment for accurate power profiling since embedded systems lack internal power monitoring.

Discussion Questions:

1. How do parallelism strategies change as we move from cloud to edge to microcontroller?
2. What are the fundamental trade-offs between parallelism and energy efficiency?
3. How should memory constraints influence parallelism design decisions?

Week 4 Key Takeaways

Parallelism Across the Computing Spectrum

Multi-Scale Parallelism Strategies:

- Data parallelism: Scales with dataset size, communication-limited
- Model parallelism: Enables large models, memory-bandwidth intensive
- Hybrid approaches: Combine benefits, increase complexity
- Microcontroller constraints: Limited to instruction-level parallelism

Energy-Efficient Processing:

- Power dominates design at embedded scale
- Duty cycling and cascading enable battery operation
- Communication costs exceed computation costs
- Hardware selection critical for energy efficiency

Memory Management:

- Flash and SRAM constraints require careful planning
- Static allocation preferred in constrained environments
- Model size vs. accuracy trade-offs essential
- Code optimization equally important as model optimization

Practical Impact:

Enable ML deployment across full spectrum from cloud to microcontroller while meeting performance, energy, and memory constraints.

Looking Ahead - Week 5 Preview

Distributed Training Systems

Next Week's Focus:

- Distributed training across heterogeneous devices
- Privacy-preserving distributed learning approaches
- Security considerations in distributed ML systems
- Federated learning for edge devices

Connection to Parallelism:

- Distributed training builds on parallelism foundations
- Network communication becomes critical bottleneck
- Heterogeneous devices require adaptive parallelism strategies
- Privacy constraints affect parallel processing design

Required Readings for Week 5:

- Book 1 - Volume 2: Distributed Training
- Book 1 - Volume 2: Collective Communication
- Book 1 - Volume 2: Fault Tolerance
- Book 1 - Volume 2: Edge Intelligence
- Book 2: Chapter 20: Privacy, Security, and Deployment